

ラストワンマイル 物資配送アプリ クラス設計書

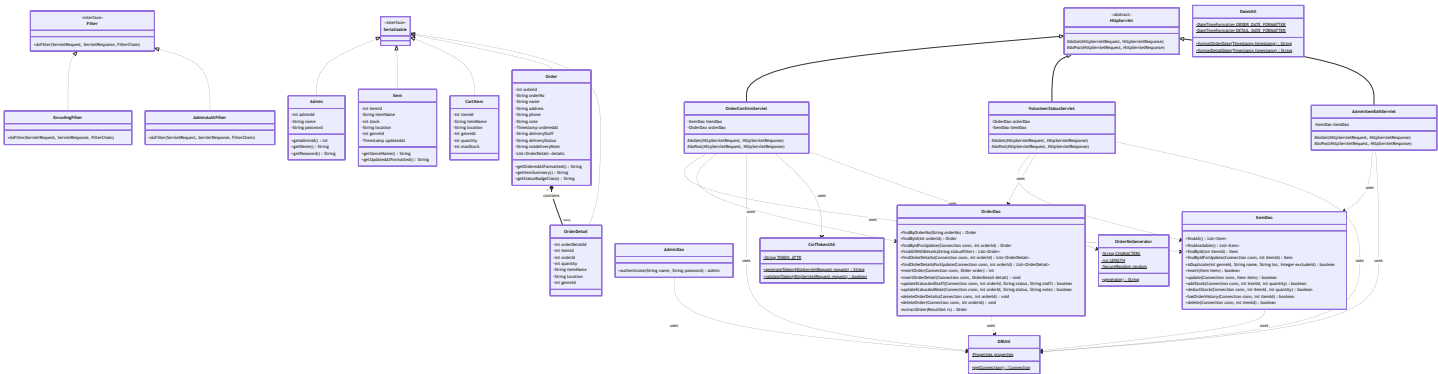
本ドキュメントは、「ラストワンマイル 物資配送アプリ」を構成するJavaクラス、データ構造、パッケージ構成、各クラスの責務および相互関係について定義するものです。

1. アーキテクチャおよびパッケージ構成概要

本システムは、保守性・堅牢性・拡張性に優れたWebアプリケーションを実現するため、Jakarta EE標準仕様に基づく **MVC (Model-View-Controller) / DAO (Data Access Object)** パターンを採用しています。

```
project/src/main/java
├── filter/           【Filter層】 共通リクエスト処理・セキュリティ制御
│   ├── AdminAuthFilter.java      (管理者認証チェックフィルタ)
│   └── EncodingFilter.java        (文字エンコーディングフィルタ)
├── servlet/         【Control層】 HTTPリクエスト制御・画面遷移ハンドラ
│   ├── TopServlet.java
│   ├── OrderItemsServlet.java
│   ├── OrderAddressServlet.java
│   ├── OrderConfirmServlet.java
│   ├── OrderCompleteServlet.java
│   ├── OrderStatusServlet.java
│   ├── OrderCancelServlet.java
│   ├── VolunteerListServlet.java
│   ├── VolunteerDetailServlet.java
│   ├── VolunteerStatusServlet.java
│   ├── AdminLoginServlet.java
│   ├── AdminLogoutServlet.java
│   ├── AdminMenuServlet.java
│   ├── AdminItemListServlet.java
│   ├── AdminItemAddServlet.java
│   ├── AdminItemEditServlet.java
│   ├── AdminItemStockServlet.java
│   ├── AdminItemDeleteServlet.java
│   ├── AdminOrderListServlet.java
│   ├── AdminOrderDetailServlet.java
│   └── AdminOrderDeleteServlet.java
├── model/           【Model層】 ビジネスエンティティ・値オブジェクト
│   ├── Admin.java
│   ├── Item.java
│   ├── CartItem.java
│   ├── Order.java
│   └── OrderDetail.java
├── dao/             【Data Access層】 データベースCRUD操作・SQL発行
│   ├── AdminDao.java
│   ├── ItemDao.java
│   └── OrderDao.java
└── util/            【Utility層】 共通ヘルパー・インフラストラクチャ
    ├── DBUtil.java
    ├── DateUtil.java
    ├── OrderNoGenerator.java
    └── CsrfTokenUtil.java
```

以下に、システムを構成する主要なクラス群の静的な関係性を示します。



3.1 model パッケージ

すべてのエンティティおよび値オブジェクトは `java.io.Serializable` を実装し、HTTPセッション格納に対応しています。

(1) model.Admin

管理者情報を保持するモデルクラスです。

- **フィールド:**
 - `int adminId`: 管理者ID (主キー)
 - `String name`: 管理者ログインID (氏名)
 - `String password`: ログインパスワード
- **メソッド:**
 - コンストラクタ (デフォルト、全項目)
 - 各種 `getter / setter`

(2) model.Item

救援物資マスタのエンティティクラスです。

- **フィールド:**
 - `int itemId`: 品目ID (主キー、serial)
 - `String itemName`: 品目名 (最大100文字)
 - `int stock`: 在庫数 (0以上の整数)
 - `String location`: 保管場所 (例: "A-1" など最大10文字)
 - `int genreId`: カテゴリID (1:食料・飲料, 2:衛生・医療用品, 3:生活・日用品, 4:防寒・睡眠・衣類, 5:インフラ・環境整備)
 - `Timestamp updatedAt`: 最終更新日時
- **ビジネスメソッド:**
 - `getGenreName()`: `genreId` に応じた日本語名称を返却 (Java 21のswitch式を使用)。
 - `getUpdatedAtFormatted()`: `DateUtil.formatDetailDate(updatedAt)` を呼び出し、`yyyy-MM-dd HH:mm:ss` 形式の文字列を返却。

(3) model.CartItem

申請画面でユーザーが一時選択した物資および数量をセッション保持するためのDTOです。

- フィールド:
 - int itemId : 対象品目ID
 - String itemName : 品目名
 - String location : 保管場所
 - int genreId : カテゴリID
 - int quantity : 要請数量 (1以上の整数)
 - int maxStock : 選択時点での最大在庫数

(4) model.Order

救援物資の配送注文情報を保持するエンティティクラスです。

- フィールド:
 - int orderId : 注文連番ID (主キー、serial)
 - String orderNo : 照会用注文番号 (英数字7桁、ユニーク)
 - String name : 注文者氏名 (最大20文字)
 - String address : お届け先住所・避難所 (最大255文字)
 - String phone : 連絡先電話番号 (11桁以内の数字)
 - String note : 備考・要望 (テキスト)
 - Timestamp orderedAt : 申請受付日時
 - String deliveryStaff : 配送担当ボランティア氏名
 - String deliveryStatus : 配送ステータス ("未対応", "対応中", "配送不可", "完了")
 - String notdeliveryNote : 配送不可時の詳細理由
 - List<OrderDetail> details : 注文に含まれる明細リスト
- ビジネスメソッド:
 - getOrderedAtFormatted() : DateUtil.formatOrderDate(orderedAt) により yyyy-MM-dd HH:mm 形式に整形。
 - getItemSummary() : 明細を走査し、「品名 × 数量 個, ...」のサマリ文字列を動的生成。
 - getStatusBadgeClass() : ステータスに応じたCSSバッジクラス名 (badge-warning , badge-info , badge-success , badge-danger) を返却。

(5) model.OrderDetail

注文と物資を紐付ける中間明細エンティティです。

- フィールド:
 - int orderDetailId : 注文明細ID (主キー、serial)
 - int itemId : 物資ID
 - int orderId : 注文ID (外部キー)
 - int quantity : 注文数量
 - String itemName : 物資名 (JOIN取得用)
 - String location : 保管場所 (JOIN取得用)
 - int genreId : カテゴリID (JOIN取得用)

3.2 dao パッケージ

データベース (PostgreSQL) との全通信を集約し、PreparedStatementを用いたSQLインジェクション対策とトランザクション (コネクション外部注入) に対応しています。

(1) dao.AdminDao

- Admin authenticate(String name, String password) throws SQLException
 - 管理者テーブル（admin）より名前とパスワードが完全一致するレコードを検索・取得します。

(2) dao.ItemDao

- List<Item> findAll() throws SQLException: 全品目をID昇順で取得。
- List<Item> findAvailable() throws SQLException: 在庫が1以上の品目をカテゴリ・ID昇順で取得（申請者向け）。
- Item findById(int itemId) throws SQLException: 単一品目の取得。
- Item findByIdForUpdate(Connection conn, int itemId) throws SQLException: 行排他ロック（SELECT ... FOR UPDATE）付きの品目取得。
- boolean isDuplicate(int genreId, String itemName, String location, Integer excludeItemId) throws SQLException: 同一カテゴリ・品名・保管場所の重複存在チェック。編集時は自身のIDを除外可能。
- boolean insert(Item item) throws SQLException: 物資新規登録。
- boolean update(Connection conn, Item item) throws SQLException: 物資情報更新。
- boolean addStock(Connection conn, int itemId, int quantity) throws SQLException: 在庫加算（入荷、注文取消、配送不可時）。
- boolean deductStock(Connection conn, int itemId, int quantity) throws SQLException: 在庫減算（注文引当時）。
- boolean hasOrderHistory(Connection conn, int itemId) throws SQLException: 注文明細への存在チェック（削除制限検証）。
- boolean delete(Connection conn, int itemId) throws SQLException: 物資の物理削除。

(3) dao.OrderDao

- Order findByIdOrderNo(String orderNo) throws SQLException: 注文番号から注文情報および紐付く明細一覧を取得。
- Order findById(int orderId) throws SQLException: 注文IDから注文情報および明細一覧を取得。
- Order findByIdForUpdate(Connection conn, int orderId) throws SQLException: 注文情報の行排他ロック取得（ステータス変更競合防止）。
- List<Order> findAllWithDetails(String statusFilter) throws SQLException: タブ条件（"all", "undelivery", "delivery", "complete", "unable"）に応じた注文一覧と明細を1クエリ（LEFT JOIN）で効率的に取得。
- List<OrderDetail> findOrderDetails(Connection conn, int orderId) throws SQLException: 明細一覧の取得。
- List<OrderDetail> findOrderDetailsForUpdate(Connection conn, int orderId) throws SQLException: 明細情報の行排他ロック取得。
- int insertOrder(Connection conn, Order order) throws SQLException: 注文基本情報の登録（RETURNING order_id で採番IDを取得）。
- void insertOrderDetail(Connection conn, OrderDetail detail) throws SQLException: 注文明細の登録。
- boolean updateStatusAndStaff(Connection conn, int orderId, String status, String staff) throws SQLException: 配送ステータスおよび担当者氏名の更新。
- boolean updateStatusAndNote(Connection conn, int orderId, String status, String notdeliveryNote) throws SQLException: 配送ステータスおよび配送不可理由の更新。
- void deleteOrderDetails(Connection conn, int orderId) throws SQLException: 注文明細の削除。
- void deleteOrder(Connection conn, int orderId) throws SQLException: 注文の削除。

3.3 servlet パッケージ

すべてのサーブレットは jakarta.servlet.http.HttpServlet を継承し、web.xml によりマッピングされています。

クラス名	URLパターン	主要メソッド	責務・処理概要
TopServlet	/top , ""	doGet , doPost	トップ画面の表示。 管理者ログインセッションの有無に応じたUI制御。

クラス名	URLパターン	主要メソッド	責務・処理概要
OrderItemsServlet	/order	doGet , doPost	物資一覧のカテゴリ表示、 要請数量の入力受付とセッション保持。
OrderAddressServlet	/order/address	doGet , doPost	お届け先・連絡先のバリデーション（桁数、 文字種）およびセッション保持。
OrderConfirmServlet	/order/confirm	doGet , doPost	CSRF検証、トランザクション開始、在庫引当 （ FOR UPDATE ）、注文番号生成、注文登録。
OrderCompleteServlet	/order/complete	doGet	発番された注文番号の1回限り表示 （F5更新耐性）。
OrderStatusServlet	/order/status	doGet , doPost	注文番号による配送状況検索・表示。
OrderCancelServlet	/order/cancel	doGet , doPost	未対応注文の申請取り消し、 在庫の自動棚戻しトランザクション。
VolunteerListServlet	/volunteer/list	doGet	ボランティア向け配送一覧表示 （タブによるステータス絞り込み）。
VolunteerDetailServlet	/volunteer/detail	doGet	配送詳細表示（依頼品、配送先、 ステータス変更UI）。
VolunteerStatusServlet	/volunteer/status	doGet , doPost	配送担当者確定（排他制御）、配送完了、配送不可 （理由登録・在庫返還）。
AdminLoginServlet	/admin/login	doGet , doPost	管理者ID・パスワードによる認証、セッション loginAdmin 発行。
AdminLogoutServlet	/admin/logout	doGet , doPost	管理者セッションの破棄 （ session.invalidate() ）。
AdminMenuServlet	/admin/menu	doGet	管理者メニュー画面表示。
AdminItemListServlet	/admin/items	doGet	管理者用物資一覧表示（全件表示、在庫確認）。
AdminItemAddServlet	/admin/item/add	doGet , doPost	物資新規登録（CSRF検証、品名・ 保管場所の重複チェック）。
AdminItemEditServlet	/admin/item/edit	doGet , doPost	物資情報更新（排他ロック、重複チェック）。
AdminItemStockServlet	/admin/item/stock	doGet , doPost	物資の在庫入荷加算処理。
AdminItemDeleteServlet	/admin/item/delete	doGet , doPost	物資削除（注文履歴存在時は整合性エラー表示）。
AdminOrderListServlet	/admin/orders	doGet	管理者用全注文一覧表示（5ステータスタブ）。
AdminOrderDetailServlet	/admin/order/detail	doGet	管理者用注文詳細表示、削除ダイアログ制御。
AdminOrderDeleteServlet	/admin/order/delete	doGet , doPost	未対応注文の管理者強制削除、在庫自動返還。

3.4 filter パッケージ

(1) filter.EncodingFilter

- マッピング: /* (全リクエスト)
- 処理内容:
 - request.setCharacterEncoding("UTF-8")
 - response.setCharacterEncoding("UTF-8")
 - 文字化けを防止し、アプリケーション全体のUTF-8入出力を保証します。

(2) filter.AdminAuthFilter

- マッピング: /admin/*
- 処理内容:
 - /admin/login へのリクエストは認証不要としてスルー。
 - その他の /admin/* へのアクセス時、セッションの loginAdmin 属性の存在をチェック。存在しない場合は /admin/login へリダイレクトして不正アクセスを遮断。

3.5 util パッケージ

(1) util.DBUtil

- クラスロード時に db.properties をクラスパスから読み込み、JDBCドライバ（org.postgresql.Driver）を初期化。
- getConnection() メソッドにより、DriverManager.getConnection() を通じてデータベースコネクションを供給。

(2) util.CsrfTokenUtil

- generateToken(HttpServletRequest request) : UUIDを用いたランダム文字列を生成してセッション属性 "csrfToken" に保存し、トークン文字列を返却。
- validateToken(HttpServletRequest request) : リクエストパラメータ "csrfToken" とセッションの値を比較。一致した場合はセッションから破棄（ワンタイム保証）して true を返却。

(3) util.DateUtil

- formatOrderDate(Timestamp timestamp) : yyyy-MM-dd HH:mm 形式にフォーマット（一覧表示用）。
- formatDetailDate(Timestamp timestamp) : yyyy-MM-dd HH:mm:ss 形式にフォーマット（詳細更新日時用）。
- java.time.format.DateTimeFormatter を使用し、スレッドセーフな高速変換を実現。

(4) util.OrderNoGenerator

- generate() : java.security.SecureRandom を使用し、誤認しやすい文字（l, O, 0, 1）を除外した英数字（ABCDEFGHIJKLMNPQRSTUVWXYZ23456789）から7桁のユニークな注文番号文字列を生成。

4. データベース設定および設定ファイル仕様

4.1 db.properties

データベース接続情報を外部定義します。

- 配置先: src/main/java/db.properties (クラスパス直下)

```

db.url=jdbc:postgresql://localhost:5432/last_onemile_db
db.user=postgres
db.password=1234
db.driver=org.postgresql.Driver

```

4.2 web.xml

Servlet 6.1 / Jakarta EE 10 仕様に基づき、フィルタ、サーブレット、マッピングおよびウェルカムファイルを定義。

- 静的リソースマッピング: /css/* -> default サーブレット
- ウェルカムファイル: index.jsp (/top へのリダイレクト)

5. クラス間相互作用マトリクス (CRUD・アクセス関係)

サーブレット	使用DAO	使用Model	使用Util	DB操作内容
TopServlet	-	Admin	-	セッション読み取り
OrderItemsServlet	ItemDao	Item , CartItem	-	findAvailable (SELECT)
OrderAddressServlet	-	CartItem , Order	-	セッション requestedItems 検証
OrderConfirmServlet	ItemDao , OrderDao	Item , Order , OrderDetail , CartItem	DBUtil , CsrfTokenUtil , OrderNoGenerator	findByIdForUpdate , deductStock , insertOrder , insertOrderDetail (TX)
OrderStatusServlet	OrderDao	Order	-	findByOrderNo (SELECT)
OrderCancelServlet	OrderDao , ItemDao	Order , OrderDetail	DBUtil	findByIdForUpdate , findOrderDetailsForUpdate , addStock , deleteOrderDetails , deleteOrder (TX)
VolunteerListServlet	OrderDao	Order	-	findAllWithDetails (SELECT)
VolunteerDetailServlet	OrderDao	Order	-	findById (SELECT)
VolunteerStatusServlet	OrderDao , ItemDao	Order , OrderDetail	DBUtil	findByIdForUpdate , updateStatusAndStaff , findOrderDetailsForUpdate , addStock , updateStatusAndNote (TX)
AdminLoginServlet	AdminDao	Admin	-	authenticate (SELECT)
AdminLogoutServlet	-	-	-	セッション破棄
AdminItemListServlet	ItemDao	Item	-	findAll (SELECT)
AdminItemAddServlet	ItemDao	Item	CsrfTokenUtil	isDuplicate , insert (INSERT)
AdminItemEditServlet	ItemDao	Item	DBUtil	findById , isDuplicate , findByIdForUpdate , update (TX)

サーブレット	使用DAO	使用Model	使用Util	DB操作内容
AdminItemStockServlet	ItemDao	Item	DBUtil	findById , findByIdForUpdate , addStock (TX)
AdminItemDeleteServlet	ItemDao	Item	DBUtil	findByIdForUpdate , hasOrderHistory , delete (TX)
AdminOrderListServlet	OrderDao	Order	-	findAllWithDetails (SELECT)
AdminOrderDetailServlet	OrderDao	Order	-	findById (SELECT)
AdminOrderDeleteServlet	OrderDao , ItemDao	Order , OrderDetail	DBUtil	findByIdForUpdate , findOrderDetailsForUpdate , addStock , deleteOrderDetails , deleteOrder (TX)